

FestFlow 홈페이지 마스터 독스

초보자도 코드와 구조를 끝까지 따라갈 수 있도록 풀어쓴 React + Spring Boot 상세 기술
문서

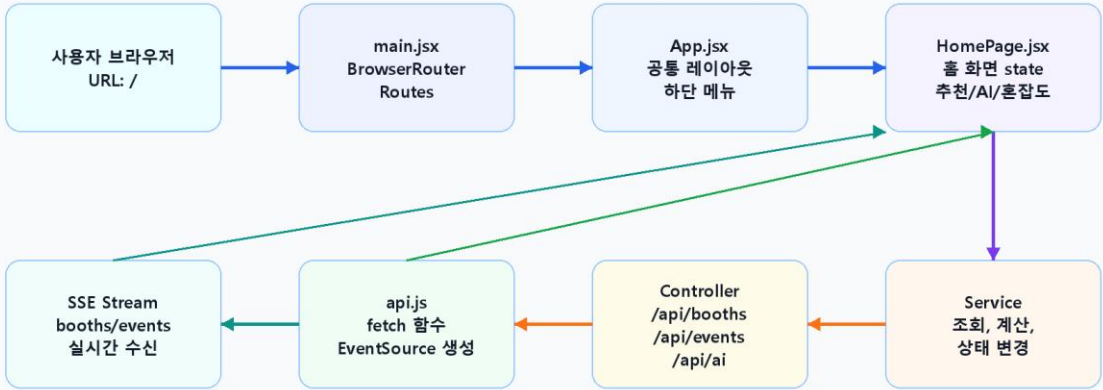
항목	내용
대상	FestFlow 홈페이지와 방문자 웹앱 전체
주요 코드	frontend/src/main.jsx, App.jsx, pages/HomePage.jsx, api.js
백엔드 범위	Controller, Service, Repository, Entity, SSE, 인증, 배포 설정
문서 성격	초보자용 마스터 독스 + 개발자용 코드 추적 가이드
작성 기준	현재 작업 폴더의 실제 소스 코드

문서 사용 방법

처음 읽는 사람은 1 장부터 8 장까지로 전체 그림을 잡고, 개발자는 9 장 이후의 HomePage, api.js, Controller/Service 연결 흐름을 따라가면 됩니다. 코드를 고칠 때는 마지막의 수정 체크리스트를 반드시 확인하세요.

FestFlow 홈페이지 코드 구조

브라우저가 React 라우터를 통해 HomePage를 렌더링하고, api.js를 거쳐 Spring Boot API와 SSE 스트림에 연결됩니다.



읽는 순서: main.jsx에서 라우트 확인 -> App.jsx에서 공통 껍데기 확인 -> HomePage.jsx에서 state/useEffect/useMemo 확인 -> api.js와 백엔드 C

그림 1. FestFlow 홈페이지 코드 구조

빠른 목차

- 1-8 장: 프로젝트 구조, 프론트/백엔드 기술 스택, 라우팅과 공통 레이아웃
- 9-19 장: HomePage.jsx 코드 상세 해부, state/useEffect/useMemo/AI 질문 처리
- 20-24 장: 홈 화면과 백엔드 API, Controller/Service, 혼잡도 계산, SSE
- 25-28 장: 주요 페이지, 브라우저 저장소, CSS, i18n
- 29-34 장: 설정, API, DB, 인증, PWA, 배포
- 35-38 장: 오류 해결, 개발 체크리스트, 코드 읽기 연습, 핵심 요약

읽기 팁

이 독스는 처음부터 끝까지 읽어도 되지만, 실제 개발 중에는 목차에서 관련 장으로 바로 이동해도 됩니다. 특히 오류 해결과 수정 체크리스트는 작업 전후로 반복해서 확인하는 용도입니다.

홈 화면 데이터 흐름

HomePage는 최초 로딩 HTTP 요청과 지속 연결 SSE 스트림을 함께 사용합니다.

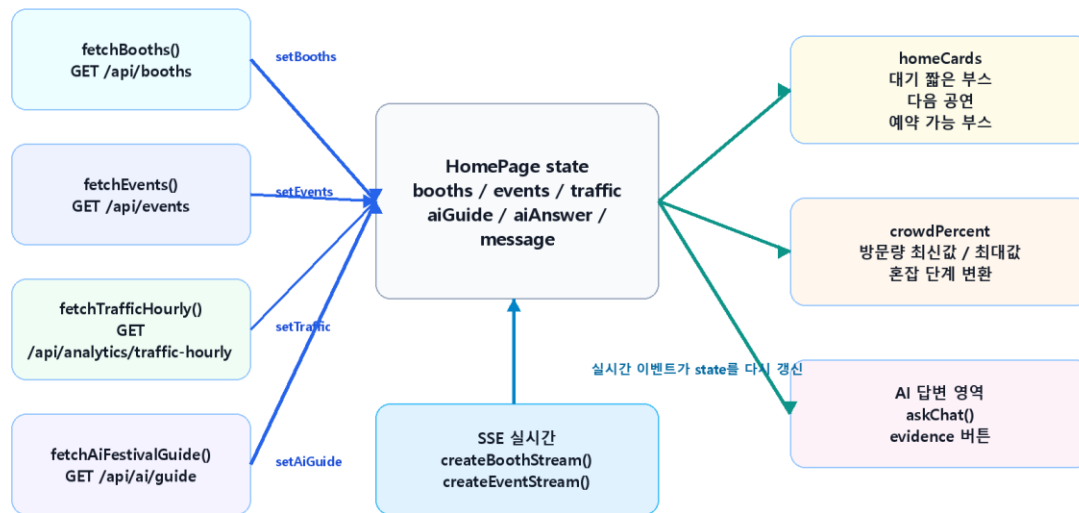


그림 2. 홈 화면 데이터 흐름

홈페이지 백엔드 연결 체인

api.js의 함수는 Controller를 호출하고, Controller는 Service로 넘기며, Service는 Repository/외부 API/SSE를 사용합니다.

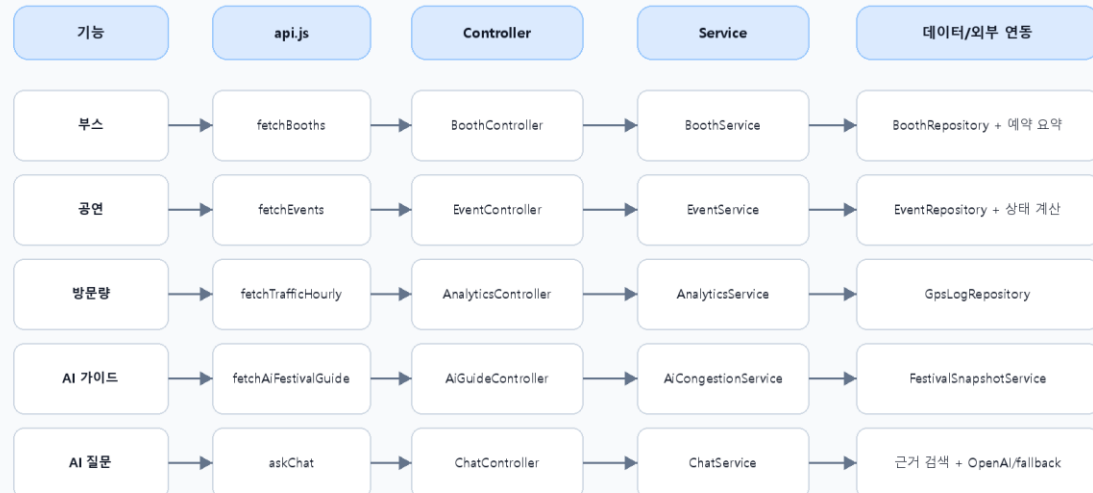


그림 3. 홈페이지 백엔드 연결 체인

이 문서는 FestFlow 홈페이지를 처음 보는 사람도 코드와 기능을 함께 이해할 수 있도록 만든 상세 기술 문서입니다. 여기서 말하는 "홈페이지"는 단순히 첫 화면(/)만이 아니라, 첫 화면에서 연결되는 방문자용 웹앱 전체와 그 뒤에서 동작하는 백엔드 API 까지 포함합니다.

작성 기준은 현재 작업 폴더의 실제 코드입니다.

- 프론트엔드 루트: frontend
- 백엔드 루트: backend
- 첫 화면 컴포넌트: frontend/src/pages/HomePage.jsx
- 라우팅 진입점: frontend/src/main.jsx
- 공통 레이아웃: frontend/src/App.jsx
- API 호출 모음: frontend/src/api.js

1. 한 문장으로 이해하기

FestFlow 는 대학교 축제 방문자가 부스, 공연, 지도, 혼잡도, 예약, 분실물, AI 안내를 모바일 화면에서 확인하고, 운영자와 관리자는 같은 데이터를 수정하고 실시간으로 반영할 수 있게 만든 React + Spring Boot 웹앱입니다.

쉽게 말하면 다음 구조입니다.

사용자 브라우저

- > React 화면
- > frontend/src/api.js
- > Spring Boot Controller
- > Service
- > Repository
- > DB
- > 다시 JSON 응답
- > React 화면 갱신

실시간 정보는 일반 HTTP 요청만으로 끝나지 않습니다. EventSource 라는 브라우저 기능을 사용해서 백엔드의 SSE 스트림을 구독합니다.

운영자가 부스 상태 변경

- > 백엔드 Service 가 DB 저장
- > StreamService 가 "booths" 이벤트 발행

- > 프론트 EventSource 가 이벤트 수신
- > 화면의 부스 목록이 즉시 바뀜

2. 초보자 용어 사전

용어	쉬운 뜻	이 프로젝트에서의 예
프론트엔드	사용자가 브라우저에서 직접 보는 화면	`frontend/src/pages/HomePage.jsx`
백엔드	화면 뒤에서 데이터 저장, 검증, 계산을 처리하는 서버	`backend/src/main/java/com/festflow/backend`
API	프론트엔드가 백엔드에게 일을 요청하는 주소	`GET /api/booths`
라우팅	URL 경로에 따라 어떤 화면을 보여줄지 정하는 것	`/events`는 `EventPage.jsx`
컴포넌트	React 화면을 구성하는 함수 단위	`HomePage`, `CongestionBadge`
state	화면이 기억하는 현재 값	`booths`, `events`, `aiAnswer`
hook	React 에서 상태와 생명주기를 쓰는 함수	`useState`, `useEffect`, `useMemo`
DTO	API 로 주고받는 데이터 모양	`BoothResponseDto`
Entity	DB 테이블과 연결되는 Java 클래스	`Booth`, `FestivalEvent`, `GpsLog`
Repository	DB 조회/저장을 쉽게 해주는 Spring Data JPA 객체	`BoothRepository`
Service	실제 규칙과 계산이 들어가는 백엔드 클래스	`BoothService`, `AiCongestionService`
SSE	서버가 브라우저에 실시간 이벤트를 밀어주는 방식	`/api/stream/booths`
PWA	모바일 앱처럼 설치 가능한 웹앱	`manifest.json`, `service-worker.js`
JWT	관리자 인증용 토큰	`Authorization: Bearer ...`
localStorage	브라우저에 값을 저장하는 저장소	즐거찾기, 예약 인증 토큰

3. 프로젝트 전체 구조

현재 폴더는 크게 이렇게 나뉩니다.

```

FestFlow
  backend
    src/main/java/com/festflow/backend
      controller      HTTP API 입구
      service          실제 비즈니스 로직
      entity           DB 테이블과 연결되는 모델
      repository       DB 접근 계층
      dto              API 요청/응답 데이터 모양
      config           CORS, Security 같은 설정
      security         JWT, 운영 키, rate limit 필터
      init             데모 데이터 초기화
    src/main/resources
      application*.properties

  frontend
    src
      main.jsx         React 앱 시작점과 라우팅
      App.jsx           공통 레이아웃과 하단 네비게이션
      api.js            백엔드 API 호출 함수 모음
      i18n.js           한/영 번역 처리
      index.css         전체 화면 스타일
      pages             실제 화면 컴포넌트들
      components        재사용 UI 컴포넌트
      utils             브라우저 저장소, 위치, 인증 유틸
      data              fallback 데이터
    public
      manifest.json
      service-worker.js
      images

```

처음 코드를 읽는다면 이 순서가 가장 좋습니다.

1. frontend/src/main.jsx 에서 URL 과 페이지 연결을 봅니다.
2. frontend/src/App.jsx 에서 공통 레이아웃과 하단 메뉴를 봅니다.
3. frontend/src/pages/HomePage.jsx 에서 첫 화면이 데이터를 어떻게 불러오는지 봅니다.
4. frontend/src/api.js 에서 어떤 백엔드 API 를 호출하는지 봅니다.
5. backend/controller/*Controller.java 에서 API 주소가 어느 서비스로 연결되는지 봅니다.
6. backend/service/*.java 에서 실제 계산과 저장 규칙을 봅니다.
7. backend/entity/*.java 에서 DB 에 어떤 값이 저장되는지 봅니다.

4. 프론트엔드 기술 스택

frontend/package.json 기준으로 주요 기술은 다음과 같습니다.

기술	역할
React 18	화면 컴포넌트 작성
React DOM	React 컴포넌트를 브라우저 DOM 에 붙임
React Router DOM 6	URL 라우팅
Vite	개발 서버와 빌드 도구
Tailwind CSS	유틸리티 CSS 기반 스타일링
Leaflet / React Leaflet	지도 화면
QRCode	예약 체크인 QR 생성
Playwright	화면 검증/테스트 도구

실행 명령은 다음과입니다.

```
cd frontend
npm install
npm run dev
```

빌드는 다음과입니다.

```
cd frontend
npm run build
```

5. 백엔드 기술 스택

backend/build.gradle 기준으로 주요 기술은 다음과 같습니다.

기술	역할
Java 17	백엔드 언어

Spring Boot 3.3.5	서버 프레임워크
Spring Web	REST API
Spring Data JPA	DB 접근
Spring Security	인증/권한
Validation	요청값 검증
MySQL Driver	MySQL 연결
PostgreSQL Driver	PostgreSQL 연결 가능성
JWT	JWT 생성/검증
AWS S3 SDK	이미지 업로드 저장소
Twilio / Solapi / Aligo	SMS 발송
OpenAI Responses/Image API 호출	AI 안내, 이미지 변환

실행 명령은 다음과 같습니다.

```
cd backend
.\gradlew.bat bootRun
```

백엔드는 기본적으로 `http://localhost:8080` 에서 실행됩니다.

6. 첫 진입점: `main.jsx`

파일: `frontend/src/main.jsx`

이 파일은 React 앱의 시작점입니다. 브라우저가 `index.html` 을 열면, 그 안의 `root DOM` 요소에 React 앱을 붙입니다.

핵심 구조는 다음과 같습니다.

```
ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <LanguageProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/" element={<App />}>
```

```

    <Route index element={lazyElement(HomePage)} />
    ...
  </Route>
</Routes>
</BrowserRouter>
</LanguageProvider>
</React.StrictMode>,
);

```

이 코드를 초보자 관점에서 풀면 다음과 같습니다.

- `document.getElementById("root")`: HTML 안의 `<div id="root"></div>`를 찾습니다.
- `ReactDOM.createRoot(...).render(...)`: 그 위치에 React 화면을 그립니다.
- `React.StrictMode`: 개발 중 잠재 문제를 더 잘 찾게 해주는 React 도구입니다.
- `LanguageProvider`: 전체 화면에서 한/영 전환을 쓸 수 있게 감싸는 Provider 입니다.
- `BrowserRouter`: 브라우저 URL 을 읽고 페이지 전환을 가능하게 합니다.
- `Routes` 와 `Route`: 어떤 URL 에서 어떤 컴포넌트를 보여줄지 정합니다.
- `App`: 모든 페이지가 공통으로 사용하는 껍데기 레이아웃입니다.
- `<Route index element={lazyElement(HomePage)} />`: / 경로에 접속하면 `HomePage` 를 보여줍니다.

6.1 lazy loading

`main.jsx` 는 대부분 페이지를 이렇게 불러옵니다.

```
const HomePage = lazy(() => import("../pages/HomePage"));
```

이것은 페이지 코드를 처음부터 전부 다운로드하지 않고, 실제로 필요할 때 가져오게 하는 방식입니다. 사용자가 `/events` 에 가지 않았는데 공연 페이지 코드까지 먼저 받을 필요는 없기 때문입니다.

6.2 라우트 목록

URL	컴포넌트	의미
<code>'/'</code>	<code>'HomePage'</code>	메인 홈
<code>'/stage-map'</code>	<code>'StageMapPage'</code>	지도/부스 탐색
<code>'/events'</code>	<code>'EventPage'</code>	공연 일정
<code>'/events/lineup'</code>	<code>'LineupPage'</code>	라인업

`/analytics`	`AnalyticsPage`	혼잡도 분석
`/booths/:id`	`BoothDetailPage`	부스 상세/예약
`/lost-found`	`LostFoundPage`	분실물
`/chat`	`ChatPage`	AI 챗봇
`/staff`	`StaffPage`	스태프 콘솔
`/more`	`MorePage`	더보기 메뉴
`/admin`	`AdminPage`	관리자
`/ops/master`	`OpsMasterPage`	통합 운영 콘솔
`/ops/booth/:id`	`OpsBoothPage`	부스 운영 콘솔
`/ai-match`	`AiMatchPage`	AI Match 사용자 화면
`/ai-match/admin`	`AiMatchAdminPage`	AI Match 관리자 화면

* 경로는 등록되지 않은 모든 URL 입니다. 이 경우 /로 돌려보냅니다.

```
<Route path="*" element={<Navigate to="/" replace />} />
```

7. 공통 레이아웃: `App.jsx`

파일: frontend/src/App.jsx

App.jsx 는 모든 페이지의 공통 껍데기입니다.

핵심 코드는 다음입니다.

```
export default function App() {
  const location = useLocation();
  const isOpsRoute = ["/admin", "/ops"].some((path) => location.pathname.startsWith(path));
  const isOpsPanelRoute = location.pathname.startsWith("/ops");
  const isAiMatchRoute = location.pathname.startsWith("/ai-match");

  return (
    <div className="app-shell festival-shell" data-route-scope={isOpsRoute ? "ops" : "public"}>
      <main className="festival-main">
        <Outlet />
      </main>
      ...
    </div>
  );
}
```

```

    </div>
  );
}

```

이 코드에서 중요한 점은 세 가지입니다.

8. `useLocation()`으로 현재 URL 을 확인합니다.
9. 현재 URL 이 운영 화면인지, AI Match 화면인지 판단합니다.
10. `<Outlet />` 자리에 실제 페이지가 들어옵니다.

7.1 `<Outlet />`의 의미

`main.jsx` 에서 이렇게 되어 있습니다.

```

<Route path="/" element={<App />}>
  <Route index element={lazyElement(HomePage)} />
  <Route path="events" element={lazyElement(EventPage)} />
</Route>

```

부모 라우트는 `App` 입니다. 자식 라우트는 `HomePage`, `EventPage` 등입니다. 이때 자식 페이지가 실제로 들어가는 위치가 `App.jsx` 의 `<Outlet />`입니다.

즉, `/events` 에 접속하면 화면 구조는 이렇게 됩니다.

```

App
├── main.festival-main
│   └── EventPage
└── bottom navigation

```

7.2 하단 네비게이션

`App.jsx` 에는 방문자용 하단 메뉴가 있습니다.

```

const navTabs = [
  { to: "/", label: "홈", icon: IconHome, end: true, match: ["/"] },
  { to: "/stage-map", label: "지도", icon: IconMapPin, match: ["/stage-map"] },
  { to: "/events", label: "공연", icon: IconCalendar, match: ["/events"] },
  { to: "/analytics", label: "혼잡도", icon: IconChart, match: ["/analytics"] },
  {
    to: "/more",
    label: "더보기",
  },
];

```

```

    icon: IconSettings,
    match: ["/more", "/ai-match", "/lost-found", "/chat", "/staff", "/admin"],
  },
];

```

각 항목은 다음 정보를 가집니다.

- to: 클릭하면 이동할 URL
- label: 화면에 보이는 글자
- icon: 사용할 아이콘 컴포넌트
- match: 어떤 URL 일 때 활성 상태로 표시할지
- end: /처럼 정확히 같은 경로일 때만 활성화해야 하는 경우 사용

운영 화면(/ops)은 별도 운영 하단 메뉴를 씁니다. AI Match 화면은 자체 경험을 가지므로 하단 네비게이션을 숨깁니다.

```

{!isOpsPanelRoute && !isAiMatchRoute && (
  <nav className="festival-bottom-nav" aria-label="주요 메뉴">
    ...
  </nav>
)}

```

8. API 호출 계층: `api.js`

파일: frontend/src/api.js

이 파일은 프론트엔드의 모든 백엔드 호출을 한곳에 모아둔 파일입니다. 페이지 컴포넌트가 직접 `fetch("http://localhost:8080/api/booths")`를 쓰지 않고, `fetchBooths()` 같은 함수를 호출하게 만드는 목적입니다.

8.1 API 기본 주소

```

const API_BASE = (
  import.meta.env.VITE_API_BASE_URL || "http://localhost:8080/api"
)
  .trim()
  .replace(/\/+$/, "");

```

이 코드는 API 기본 주소를 정합니다.

- 배포 환경에서는 VITE_API_BASE_URL 환경변수를 사용합니다.
- 환경변수가 없으면 로컬 기본값 `http://localhost:8080/api` 를 사용합니다.
- 마지막의 `/`가 여러 개 붙어 있으면 제거합니다.

예를 들어 `VITE_API_BASE_URL=https://example.com/api/` 라면 최종 값은 `https://example.com/api` 가 됩니다.

8.2 네트워크 오류 처리

```
async function fetch(...args) {
  try {
    return await globalThis.fetch(...args);
  } catch (error) {
    throw new Error(NETWORK_ERROR_MESSAGE, { cause: error });
  }
}
```

브라우저 기본 `fetch` 를 감싼 함수입니다. 서버에 아예 연결할 수 없는 경우 사용자에게 읽기 쉬운 메시지를 보여주기 위해 에러 메시지를 바꿉니다.

8.3 타임아웃 처리

```
async function fetchWithTimeout(url, options = {}, timeoutMs = 12000) {
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), timeoutMs);
  ...
}
```

일반 `fetch` 는 서버가 응답을 오래 안 해도 자동으로 끊기지 않습니다. 그래서 `AbortController` 로 일정 시간이 지나면 요청을 중단합니다.

이 기능은 특히 AI 이미지 변환, 챗봇처럼 응답이 오래 걸릴 수 있는 API 에 중요합니다.

8.4 JSON 응답 처리

```
async function parseJson(response, errorMessage) {
  if (!response.ok) {
    ...
    throw error;
  }
}
```

```

    }

    const contentType = response.headers.get("content-type") || "";
    if (!contentType.includes("application/json")) {
        return response.text();
    }
    return response.json();
}

```

이 함수는 HTTP 응답을 해석합니다.

- `response.ok` 가 아니면 에러를 만듭니다.
- 백엔드가 `{ message: "..."}` 를 보내면 그 메시지도 붙입니다.
- JSON 응답이면 `response.json()`으로 읽습니다.
- JSON 이 아니면 텍스트로 읽습니다.

8.5 관리자 인증 헤더

```

function withAuth(headers = {}) {
    const token = getAccessToken();
    if (!token) return headers;
    return { ...headers, Authorization: `Bearer ${token}` };
}

```

관리자 API 는 JWT 토큰이 필요합니다. 이 함수는 `localStorage` 에 저장된 관리자 토큰을 읽어서 `Authorization` 헤더에 붙입니다.

8.6 홈 화면에서 쓰는 API 함수

홈 화면은 주로 다음 함수를 사용합니다.

함수	HTTP 요청	용도
<code>`fetchBooths()`</code>	<code>`GET /api/booths`</code>	부스 목록
<code>`fetchEvents()`</code>	<code>`GET /api/events`</code>	공연 목록
<code>`fetchTrafficHourly()`</code>	<code>`GET /api/analytics/traffic-hourly`</code>	시간대별 방문량
<code>`fetchAiFestivalGuide()`</code>	<code>`GET /api/ai/guide`</code>	AI 축제 가이드
<code>`askChat(question)`</code>	<code>`POST /api/chat`</code>	AI 질문 답변

<code>`createBoothStream()`</code>	<code>`EventSource /api/stream/booths`</code>	부스 실시간 갱신
<code>`createEventStream()`</code>	<code>`EventSource /api/stream/events`</code>	공연 실시간 갱신

9. 홈 화면 전체 구조: `HomePage.jsx`

파일: `frontend/src/pages/HomePage.jsx`

홈 화면은 FestFlow 의 첫인상을 담당하지만, 기술적으로는 꽤 많은 일을 합니다.

홈 화면이 하는 일은 다음입니다.

11. 부스 목록을 가져옵니다.
12. 공연 목록을 가져옵니다.
13. 시간대별 방문량을 가져옵니다.
14. AI 축제 가이드를 가져옵니다.
15. 부스 실시간 스트림을 구독합니다.
16. 공연 실시간 스트림을 구독합니다.
17. 실패하면 fallback 데이터를 사용합니다.
18. 추천 카드 3 개를 계산합니다.
19. 전체 혼잡도 퍼센트를 계산합니다.
20. 사용자의 AI 질문을 챗봇 API 로 보냅니다.

9.1 import 읽기

`HomePage.jsx` 상단에는 이런 import 가 있습니다.

```
import { useEffect, useMemo, useState } from "react";
import { useNavigate } from "react-router-dom";
import {
  askChat,
  createBoothStream,
  createEventStream,
  fetchAiFestivalGuide,
  fetchBooths,
  fetchEvents,
  fetchTrafficHourly,
} from "../api";
```


이것은 홈 화면이 필요한 도구를 가져오는 부분입니다.

- `useState`: 화면이 기억할 값을 만듭니다.
- `useEffect`: 화면이 처음 뜰 때 데이터를 불러오거나 스트림을 연결합니다.
- `useMemo`: 계산 결과를 기억해서 불필요한 재계산을 줄입니다.
- `useNavigate`: 버튼을 눌렀을 때 다른 URL 로 이동합니다.
- `askChat`, `fetchBooths` 등: `api.js` 에서 만든 백엔드 호출 함수입니다.

9.2 상수

```
const EVENT_RECOMMEND_IMAGE = "/images/og-festflow.png";
const DEFAULT_AI_GUIDE = {
  headline: "지금은 대기 짧은 부스부터 둘러보는 코스를 추천합니다.",
  summary: "혼잡도, 공연 시간, 부스 대기 정보를 기준으로 지금 움직이기 좋은 동선을 골라드릴게요.",
};
const AI_HOME_PROMPTS = [
  "지금 어디 가면 좋아?",
  "안 붐비는 음식 부스 추천해줘",
  "공연 전에 들를 부스 추천해줘",
];
```

이 값들은 홈 화면에서 기본으로 쓰는 문구와 이미지입니다.

- AI API 가 실패하면 `DEFAULT_AI_GUIDE` 를 보여줍니다.
- 홈 화면 AI 추천 버튼은 `AI_HOME_PROMPTS` 배열을 반복해서 만듭니다.

9.3 작은 헬퍼 함수들

`reservationLabel(booth)`

```
function reservationLabel(booth) {
  if (booth?.reservationEnabled === false) return "현장 이용";
  const seats = Number(booth?.reservationAvailableSeats);
  if (Number.isFinite(seats) && seats > 0) return `예약 ${seats}석`;
  if (booth?.reservation) return booth.reservation;
  return "예약 확인";
}
```

이 함수는 부스 카드에 표시할 예약 문구를 만듭니다.

예를 들어:

- reservationEnabled === false 이면 "현장 이용"
- 예약 가능 좌석이 8 석이면 "예약 8 석"
- 기존 reservation 문자열이 있으면 그 문자열
- 아무 정보가 없으면 "예약 확인"

초보자가 봐야 할 포인트는 ? 입니다. booth?.reservationEnabled 는 booth 가 null 또는 undefined 여도 에러를 내지 않습니다.

`compactWaitLabel(booth)`

```
function compactWaitLabel(booth) {
  const value = booth?.estimatedWaitMinutes ?? booth?.wait;
  if (value == null || value === "") return "대기 확인 중";
  return `대기 ${String(value).replace("분", "")}분`;
}
```

이 함수는 대기 시간을 짧은 문구로 바꿉니다.

- estimatedWaitMinutes 가 있으면 우선 사용합니다.
- 없으면 fallback 데이터의 wait 를 사용합니다.
- 값이 없으면 "대기 확인 중"을 보여줍니다.
- "7 분"처럼 이미 분이 붙어 있어도 중복으로 "분분"이 되지 않게 replace("분", "")를 사용합니다.

`crowdLevel(percent)`

```
function crowdLevel(percent) {
  if (percent >= 75) return "혼잡";
  if (percent >= 45) return "보통";
  return "여유";
}
```

퍼센트를 사람이 읽기 쉬운 단계로 바꿉니다.

- 75% 이상: 혼잡
- 45% 이상: 보통
- 그 외: 여유

`compactText(value, maxLength)`

```
function compactText(value, maxLength = 90) {
  const text = cleanBrokenText(value || "");
  if (text.length <= maxLength) return text;
  return `${text.slice(0, maxLength).trim()}...`;
}
```

긴 AI 문장이나 설명이 카드 안에서 너무 길어지지 않게 자르는 함수입니다.

10. 홈 화면 state

HomePage 안에는 여러 state 가 있습니다.

```
const [booths, setBooths] = useState([]);
const [events, setEvents] = useState([]);
const [traffic, setTraffic] = useState([]);
const [aiGuide, setAiGuide] = useState(null);
const [aiGuideLoading, setAiGuideLoading] = useState(true);
const [aiQuestion, setAiQuestion] = useState("");
const [aiAnswer, setAiAnswer] = useState(null);
const [aiAskLoading, setAiAskLoading] = useState(false);
const [message, setMessage] = useState("");
```

각 state 의 의미는 다음과 같습니다.

state	처음 값	의미
`booths`	`[]`	서버에서 받은 부스 목록
`events`	`[]`	서버에서 받은 공연 목록
`traffic`	`[]`	시간대별 방문량
`aiGuide`	`null`	서버에서 받은 AI 축제 가이드
`aiGuideLoading`	`true`	AI 가이드를 불러오는 중인지
`aiQuestion`	``	사용자가 입력한 질문
`aiAnswer`	`null`	AI 챗봇 답변
`aiAskLoading`	`false`	질문 답변을 기다리는 중인지
`message`	``	화면에 보여줄 안내 메시지

useState 를 이해하는 핵심은 다음입니다.

```
const [value, setValue] = useState(initialValue);
```

- value: 현재 값
- setValue: 값을 바꾸는 함수
- initialValue: 첫 렌더링 때의 값

예를 들어 setBooths(data)를 호출하면 React 는 booths 값을 새 데이터로 바꾸고 화면을 다시 그립니다.

11. 홈 화면 최초 데이터 로딩

홈 화면이 처음 나타날 때 실행되는 핵심 코드는 useEffect 입니다.

```
useEffect(() => {
  let mounted = true;

  Promise.allSettled([
    fetchBooths(),
    fetchEvents(),
    fetchTrafficHourly(),
  ]).then(([boothResult, eventResult, trafficResult]) => {
    if (!mounted) return;
    if (boothResult.status === "fulfilled") setBooths(boothResult.value || []);
    if (eventResult.status === "fulfilled") setEvents(eventResult.value || []);
    if (trafficResult.status === "fulfilled") setTraffic(trafficResult.value || []);
    const failed = [boothResult, eventResult].some((item) => item.status === "rejected");
    setMessage(failed ? "일부 실시간 정보는 기본 안내로 표시 중입니다." : "");
  });

  ...

  return () => {
    mounted = false;
    streams.forEach((stream) => stream.close());
  };
}, []);
```

11.1 `useEffect(..., [])`

뒤에 빈 배열 []이 붙어 있습니다.

```
useEffect(() => {
```

```
...
}, []);
```

이 뜻은 "컴포넌트가 처음 화면에 나타날 때 한 번만 실행하라"입니다.

홈 화면에서는 처음 한 번 서버에서 데이터를 불러와야 하므로 이 패턴을 씁니다.

11.2 `Promise.allSettled`

```
Promise.allSettled([
  fetchBooths(),
  fetchEvents(),
  fetchTrafficHourly(),
])
```

`Promise.allSettled` 는 여러 요청을 동시에 보내고, 성공/실패 결과를 모두 받습니다.

왜 `Promise.all` 이 아니라 `Promise.allSettled` 일까요?

- `Promise.all` 은 하나만 실패해도 전체가 실패합니다.
- `Promise.allSettled` 는 하나가 실패해도 나머지 성공 결과를 쓸 수 있습니다.

홈 화면에서는 부스 API 가 실패해도 공연 API 가 성공했다면 공연은 보여주는 편이 좋습니다. 그래서 `allSettled` 가 맞습니다.

11.3 `mounted` 변수

```
let mounted = true;
...
if (!mounted) return;
...
return () => {
  mounted = false;
};
```

이 패턴은 "화면이 이미 사라졌는데 비동기 요청이 늦게 도착해서 state 를 바꾸는 문제"를 막기 위한 방어 코드입니다.

예를 들어 사용자가 홈 화면에 들어왔다가 바로 다른 페이지로 이동했는데, 뒤늦게 API 응답이 오면 이미 사라진 컴포넌트에 `setBooths` 를 호출할 수 있습니다. `mounted` 는 그런 상황을 줄이기 위한 안전장치입니다.

12. 홈 화면 AI 가이드 로딩

```
fetchAiFestivalGuide()
  .then((data) => {
    if (mounted) setAiGuide(data || null);
  })
  .catch(() => {
    if (mounted) setAiGuide(null);
  })
  .finally(() => {
    if (mounted) setAiGuideLoading(false);
  });
```

이 코드는 `/api/ai/guide` 를 호출합니다.

성공하면 `aiGuide` 에 데이터를 저장합니다. 실패하면 `null` 로 둡니다. `finally` 는 성공하든 실패하든 실행되므로 로딩 상태를 `false` 로 바꿉니다.

이후 화면에서는 다음 코드로 fallback 을 결정합니다.

```
const visibleAiGuide = aiGuide || DEFAULT_AI_GUIDE;
```

즉:

- 서버 AI 가이드가 있으면 서버 데이터를 보여줍니다.
- 없으면 기본 문구를 보여줍니다.

13. 홈 화면 실시간 스트림

홈 화면은 부스와 공연 스트림을 구독합니다.

```
const boothStream = createBoothStream();
boothStream.addEventListener("booths", (event) => {
  try {
    const next = JSON.parse(event.data);
    if (Array.isArray(next)) setBooths(next);
  } catch {
    // Ignore malformed stream payloads.
  }
});
streams.push(boothStream);
```

이 코드를 쉽게 풀면 다음과 같습니다.

21. `createBoothStream()`이 `new EventSource("/api/stream/booths")`를 만듭니다.

22. 백엔드가 "booths"라는 이름의 이벤트를 보내면 콜백이 실행됩니다.

23. `event.data`는 문자열이므로 `JSON.parse`로 배열로 바꿉니다.

24. 배열이면 `setBooths(next)`로 화면 데이터를 갱신합니다.

공연 스트림도 같은 방식입니다.

```
const eventStream = createEventStream();
eventStream.addEventListener("events", (event) => {
  ...
});
```

중요한 정리:

스트림 함수	백엔드 경로	이벤트 이름	화면에서 바꾸는 state
<code>`createBoothStream()`</code>	<code>`/api/stream/booths`</code>	<code>`booths`</code>	<code>`booths`</code>
<code>`createEventStream()`</code>	<code>`/api/stream/events`</code>	<code>`events`</code>	<code>`events`</code>

14. fallback 데이터

홈 화면은 서버 데이터가 없으면 fallback 데이터를 씁니다.

```
const boothSource = booths.length ? booths : fallbackBooths;
const eventSource = events.length ? events : fallbackEvents;
```

이 말은 다음과 같습니다.

- 서버 부스 목록이 있으면 `booths`
- 없으면 `fallbackBooths`
- 서버 공연 목록이 있으면 `events`
- 없으면 `fallbackEvents`

fallback 데이터는 `frontend/src/data/festivalUiData.js`에 있습니다.

예를 들어 fallback 부스에는 다음 같은 값이 들어 있습니다.

```
{
```

```

id: 1,
name: "공대 야시장",
category: "푸드",
description: "닭꼬치, 감자튀김, 컵밥을 빠르게 받을 수 있는 메인 푸드 부스",
latitude: 37.2832,
longitude: 127.0451,
wait: "7 분",
estimatedWaitMinutes: 7,
congestion: "보통",
reservationAvailableSeats: 8,
reservationEnabled: true,
}

```

이 fallback 은 서버가 꺼져 있거나 데이터가 비어 있어도 화면이 완전히 비어 보이지 않게 만드는 역할을 합니다.

15. 추천 카드 계산: `homeCards`

홈 화면의 "지금 추천" 3 개 카드는 useMemo 로 계산됩니다.

```

const homeCards = useMemo(() => {
  const sortedBooths = [...boothSource].sort(
    (a, b) => (Number(a.estimatedWaitMinutes) || 0) - (Number(b.estimatedWaitMinutes) || 0),
  );
  const nextEvent = [...eventSource]
    .filter((event) => event.startTime)
    .sort((a, b) => new Date(a.startTime) - new Date(b.startTime))[0];
  const firstBooth = sortedBooths[0];
  ...
}, [boothSource, eventSource]);

```

15.1 `useMemo`의 의미

useMemo 는 계산 결과를 기억합니다. boothSource 나 eventSource 가 바뀔 때만 다시 계산합니다.

홈 카드 계산은 정렬과 검색을 하므로, 매번 렌더링할 때 반복하지 않도록 useMemo 를 사용합니다.

15.2 첫 번째 카드

첫 번째 카드는 대기 시간이 가장 짧은 부스입니다.

```

const sortedBooths = [...boothSource].sort(
  (a, b) => (Number(a.estimatedWaitMinutes) || 0) - (Number(b.estimatedWaitMinutes) || 0),

```



```
);
const firstBooth = sortedBooths[0];
```

estimatedWaitMinutes 가 작은 순서로 정렬하고 첫 번째를 고릅니다.

15.3 두 번째 카드

두 번째 카드는 가장 가까운 다음 공연입니다.

```
const nextEvent = [...eventSource]
  .filter((event) => event.startTime)
  .sort((a, b) => new Date(a.startTime) - new Date(b.startTime))[0];
```

공연 시작 시간이 있는 이벤트만 골라 시간순으로 정렬하고 첫 번째를 고릅니다.

주의할 점: 이 코드는 현재 시간 이후 공연만 필터링하지는 않습니다. 단순히 startTime 이 있는 공연을 시간순으로 고릅니다. 공연 데이터가 과거/미래 섞여 있을 때는 백엔드의 상태 계산이나 데이터 정렬에 의존합니다.

15.4 세 번째 카드

세 번째 카드는 예약 가능한 부스입니다.

```
const reservable =
  boothSource.find(
    (booth) =>
      booth.id !== firstBooth?.id &&
      booth.reservationEnabled !== false &&
      Number(booth.reservationAvailableSeats) > 0,
  ) ||
  sortedBooths.find((booth) => booth.id !== firstBooth?.id) ||
  boothSource[0];
```

우선순위는 다음과 같습니다.

25. 첫 번째 추천 부스와 다른 부스
26. 예약 기능이 꺼져 있지 않음
27. 예약 가능 좌석이 1 석 이상
28. 그런 부스가 없으면 첫 번째 부스와 다른 아무 부스
29. 그래도 없으면 전체 첫 부스

16. 전체 혼잡도 계산: `crowdPercent`

```
const crowdPercent = useMemo(() => {
  if (!traffic.length) return 55;
  const latest = Number(traffic[traffic.length - 1]?.count) || 0;
  const max = Math.max(1, ...traffic.map((item) => Number(item.count) || 0));
  return Math.min(99, Math.max(1, Math.round((latest / max) * 100)));
}, [traffic]);
```

이 계산은 시간대별 방문량 중 가장 높은 값 대비 현재 최신 방문량이 어느 정도인지 퍼센트로 보여줍니다.

예를 들어 `traffic` 이 다음과 같다고 가정합니다.

```
[
  { hour: "18:00", count: 20 },
  { hour: "19:00", count: 50 },
  { hour: "20:00", count: 25 }
]
```

- 최신 값은 25 입니다.
- 최대 값은 50 입니다.
- $25 / 50 * 100 = 50\%$
- 화면에는 50%, 단계는 보통이 됩니다.

데이터가 없으면 기본값으로 55%를 씁니다.

17. AI 질문 처리: `handleAiAsk`

홈 화면 AI 질문은 이 함수가 처리합니다.

```
async function handleAiAsk(question = aiQuestion) {
  const nextQuestion = question.trim();
  if (!nextQuestion || aiAskLoading) return;

  setAiAskLoading(true);
  setAiQuestion("");
  setAiAnswer({
    question: nextQuestion,
    answer: "AI 가 축제 데이터를 확인하면서 답변을 만드는 중입니다.",
    evidence: [],
    pending: true,
  });
  try {
```

```

const result = await askChat(nextQuestion);
setAiAnswer({
  question: nextQuestion,
  answer: result.answer,
  evidence: Array.isArray(result.evidence) ? result.evidence : [],
  pending: false,
});
} catch (error) {
  setAiAnswer({
    question: nextQuestion,
    answer: error.message || "AI 답변을 불러오지 못했습니다.",
    evidence: [],
    pending: false,
  });
} finally {
  setAiAskLoading(false);
}
}

```

처리 순서는 다음과 같습니다.

30. 질문 문자열의 앞뒤 공백을 제거합니다.
31. 질문이 비어 있으면 아무것도 하지 않습니다.
32. 이미 질문 중이면 중복 요청을 막습니다.
33. 로딩 상태를 켭니다.
34. 입력창을 비웁니다.
35. 임시 답변 "AI 가 ... 만드는 중입니다."를 표시합니다.
36. askChat(nextQuestion)으로 백엔드 /api/chat 에 요청합니다.
37. 성공하면 실제 답변과 근거를 저장합니다.
38. 실패하면 에러 메시지를 답변 영역에 표시합니다.
39. 마지막에 로딩 상태를 끕니다.

이 함수는 사용자 경험 측면에서 중요합니다. 서버 응답이 늦어도 화면이 멈춘 것처럼 보이지 않고 "생성 중" 메시지가 먼저 보이기 때문입니다.

18. AI 근거 클릭 경로: `evidencePath`

```

function evidencePath(item) {
  if (!item?.id) return null;
  if (item.type === "booth" || item.type === "ai_recommendation" || item.type === "ai_warning") {
    return `/booths/${item.id}`;
  }
  if (item.type === "event") return "/events";
}

```

```

    if (item.type === "lost_item") return "/lost-found";
    return null;
}

```

AI 답변에는 evidence 라는 근거 목록이 올 수 있습니다. 이 함수는 근거 타입에 따라 사용자를 어디로 보내야 할지 결정합니다.

evidence type	이동 경로
`booth`	`/booths/{id}`
`ai_recommendation`	`/booths/{id}`
`ai_warning`	`/booths/{id}`
`event`	`/events`
`lost_item`	`/lost-found`
그 외	이동 없음

19. 홈 화면 JSX 구조

홈 화면의 최종 렌더 구조는 크게 네 덩어리입니다.

```

return (
  <section className="uni-page uni-home-page reference-home-page">
    <header className="home-hero-card">...</header>
    {message && <p className="app-inline-note">{message}</p>}
    <section className="uni-card ai-guide-card">...</section>
    <section className="uni-section">...</section>
    <section className="uni-card crowd-summary-card">...</section>
  </section>
);

```

19.1 히어로 영역

히어로 영역은 앱 이름, 공지 버튼, 핵심 문구, 빠른 이동 버튼 3 개를 보여줍니다.

```

<button type="button" onClick={() => navigate("/events")}>
  <IconMusic className="h-5 w-5" />
  <span>공연 보기</span>
</button>

```

`navigate("/events")`는 React Router 의 페이지 이동 함수입니다. 브라우저 전체를 새로고침하지 않고 React 내부에서 화면만 바꿉니다.

19.2 AI 축제 가이드 카드

이 영역은 AI 가이드의 제목/요약, 빠른 질문 버튼, 직접 질문 입력창, 답변 영역을 포함합니다.

```
<strong>{compactText(visibleAiGuide.headline, 42)}</strong>
<p>{compactText(visibleAiGuide.summary, 82)}</p>
```

긴 문구가 카드 밖으로 튀어나가지 않도록 `compactText` 를 사용합니다.

19.3 지금 추천 카드

```
{homeCards.slice(0, 3).map((item, index) => (
  <button
    key={`_${item.type}-${item.id} || item.title`}
    type="button"
    className={`recommend-card recommend-card--${cardTone(index)}`}
    ...
  >
    <span>{item.tag}</span>
    <strong>{item.title}</strong>
    <small>{item.caption}</small>
  </button>
)}}}
```

`map` 은 배열을 반복해서 React 요소를 만드는 방법입니다. `homeCards` 배열의 각 항목이 추천 카드 하나가 됩니다.

클릭하면:

- 공연 카드면 `/events`
- 부스 카드면 `/booths/{id}`
- id 가 없으면 `/stage-map`

으로 이동합니다.

19.4 실시간 혼잡도 카드

```
<section className="uni-card crowd-summary-card" onClick={() => navigate("/analytics")}>
```

```
...
<strong>{crowdPercent}%</strong>
...
</section>
```

카드 전체를 클릭하면 /analytics 로 이동합니다.

20. 홈 화면과 백엔드 연결 흐름

홈 화면의 데이터 흐름을 API 별로 따라가면 다음과 같습니다.

20.1 부스 목록

```
HomePage.jsx
  fetchBooths()
api.js
  GET /api/booths
BoothController.java
  getBooths()
BoothService.java
  getAllBooths()
BoothRepository
  findAll()
BoothService
  toDto()
React
  setBooths(...)
```

백엔드 코드는 다음 구조입니다.

```
@GetMapping
public List<BoothResponseDto> getBooths() {
    return boothService.getAllBooths();
}
```

서비스에서는 부스를 표시 순서 기준으로 정렬합니다.

```
public List<BoothResponseDto> getAllBooths() {
    return boothRepository.findAll().stream()
        .sorted(Comparator.comparing(Booth::getDisplayOrder).thenComparing(Booth::getId))
        .map(this::toDto)
        .toList();
}
```

즉, 홈 화면의 부스 순서는 DB 의 displayOrder 와 id 에 의해 결정됩니다.

20.2 공연 목록

```

HomePage.jsx
  fetchEvents()
api.js
  GET /api/events
EventController.java
  getEvents()
EventService.java
  getAllEvents()
EventRepository
  findAll()
EventService
  resolveStatus()
React
  setEvents(...)

```

EventService 는 단순히 DB 값을 그대로 보내지 않고 현재 시간 기준으로 상태를 계산합니다.

```

private String resolveStatus(FestivalEvent event, LocalDateTime now) {
    if (event.getStatusOverride() != null && !event.getStatusOverride().isBlank()) {
        ...
        return override;
    }

    String status;
    if (now.isBefore(event.getStartTime())) {
        status = "예정";
    } else if (now.isAfter(event.getEndTime())) {
        status = "종료";
    } else {
        status = "진행중";
    }
    ...
    return status;
}

```

운영자가 statusOverride 를 설정하면 그 값이 우선입니다. 없으면 현재 시간이 시작 전인지, 종료 후인지, 진행 중인지에 따라 상태를 계산합니다.

20.3 방문량

```

HomePage.jsx
  fetchTrafficHourly()
api.js
  GET /api/analytics/traffic-hourly
AnalyticsController.java
  trafficHourly()
AnalyticsService.java
  trafficHourly()

```

```
GpsLogRepository
    findByCreatedAtAfter(now - 24h)
React
    setTraffic(...)
```

`AnalyticsService.trafficHourly()`는 최근 24 시간 GPS 로그를 시간 단위로 묶습니다.

```
LocalDateTime from = LocalDateTime.now().minusHours(24);
List<GpsLog> logs = gpsLogRepository.findByCreatedAtAfter(from);
```

그리고 MM-dd HH:00 형식의 키로 카운트를 만듭니다.

20.4 AI 축제 가이드

```
HomePage.jsx
    fetchAiFestivalGuide()
api.js
    GET /api/ai/guide
AiGuideController.java
    guide()
AiCongestionService.java
    guide()
FestivalSnapshotService
    current()
AiCongestionService
    analyze(...)
React
    setAiGuide(...)
```

컨트롤러는 얇습니다.

```
@GetMapping("/guide")
public AiFestivalGuideDto guide() {
    return aiCongestionService.guide();
}
```

실제 로직은 `AiCongestionService`에 있습니다. 이 서비스는 부스별로 위험 점수를 계산합니다.

위험 점수에는 다음 요소가 반영됩니다.

요소	의미
주변 감지 인원	GPS 로그 기반 현재 밀집도
예상 대기 시간	운영자가 입력한 부스 대기 시간

활성 예약 수	예약/체크인으로 인한 테이블 점유
예약 가능 좌석	예약 가능성이 낮으면 위험 증가
재고	재고 부족이면 위험 증가
30 분 내 공연 시작	공연 전 이동량 증가 가능성

이 점수에 따라 AI 가이드의 `recommendedNow`, `avoidNow`, `recommendedLater` 가 만들어집니다.

20.5 AI 질문

```

HomePage.jsx
  askChat(question)
api.js
  POST /api/chat
ChatController.java
  chat(...)
ChatService.java
  answer(question)
ChatService
  retrieveEvidence(question)
OpenAI Responses API 또는 fallback
React
  setAiAnswer(...)

```

ChatService 는 먼저 질문 의도를 분석해 근거 데이터를 찾습니다.

- 부스 질문이면 부스와 혼잡도 근거를 찾습니다.
- 공연 질문이면 공연 근거를 찾습니다.
- 분실물 질문이면 분실물 근거를 찾습니다.
- 혼잡도 질문이면 AI 추천 근거도 붙입니다.
- 근거가 없으면 정적 FAQ 근거를 사용합니다.

OpenAI API 키가 없으면 자체 fallback 답변을 만듭니다.

```

if (apiKey == null || apiKey.isBlank()) {
    List<String> warnings = new ArrayList<>(retrieval.warnings());
    warnings.add("AI API 키가 설정되지 않아 기본 안내로 응답했습니다.");
    return new ChatResponseDto(buildFallbackAnswer(question, retrieval), confidence,
        retrieval.evidence(), wa...
}

```

이 덕분에 OpenAI 키가 없어도 화면이 완전히 고장나지 않습니다.

21. 백엔드 Controller 구조

Controller 는 HTTP 요청 주소를 받는 입구입니다. 대체로 직접 계산하지 않고 Service 를 호출합니다.

Controller	기본 경로	역할
`BoothController`	`/api/booths`	부스 목록, 상세, 혼잡도, 예약
`EventController`	`/api/events`	공연 목록
`AnalyticsController`	`/api/analytics`	방문량, 인기 부스, 히트맵, 대시보드
`AiGuideController`	`/api/ai`	AI 축제 가이드, 혼잡 예측, 방문자 가이드
`ChatController`	`/api/chat`	AI 챗봇
`NoticeController`	`/api/notices`	활성 공지
`LostItemController`	`/api/lost-items`	분실물
`StreamController`	`/api/stream`	SSE 실시간 스트림
`AuthController`	`/api/auth`	관리자 로그인
`Admin*Controller`	`/api/admin/**`	관리자 기능
`OpsController`	`/api/ops`	운영 콘솔
`StaffController`	`/api/staff`	스태프 콘솔
`AiMatchController`	`/api/ai-match`	AI Match 사용자 기능

초보자가 Controller 를 읽을 때는 다음 질문을 하면 됩니다.

40. 이 API 의 URL 은 무엇인가?
41. HTTP 메서드는 GET, POST, PUT, DELETE 중 무엇인가?
42. 요청 body 가 필요한가?
43. 경로 변수(@PathVariable)가 있는가?
44. 헤더(@RequestHeader)가 필요한가?
45. 어느 Service 메서드를 호출하는가?

22. 백엔드 Service 구조

Service 는 실제 규칙이 있는 곳입니다.

Service	주요 역할
`BoothService`	부스 CRUD, 부스 DTO 변환, 혼잡도 계산
`EventService`	공연 상태 계산, 공연 실시간 발행
`AnalyticsService`	GPS 로그 기반 방문량/히트맵/혼잡 대시보드
`AiCongestionService`	부스별 혼잡 위험 점수와 AI 추천 계산
`PublicAiGuideService`	페이지별 방문자 AI 가이드 생성
`ChatService`	질문 근거 검색, OpenAI 또는 fallback 답변
`NoticeService`	공지 CRUD 와 활성 공지
`ReservationService`	부스 예약, 체크인, 만료, 테이블 상태
`ReservationAuthService`	휴대폰 인증과 예약 토큰
`StreamService`	SSE 연결 목록 관리와 이벤트 발행
`StaffService`	스태프 로그인, 상태, 위치
`LostItemService`	분실물 등록/수정/상태 변경
`UploadStorageService`	로컬/s3 파일 업로드
`AuthService`	관리자 로그인
`JwtService`	JWT 발급/검증

23. 혼잡도 계산 이해하기

`BoothService.getCongestionByBoothId` 는 부스 주변 GPS 로그를 이용합니다.

핵심 상수:

```
private static final double BOOTH_RADIUS_METERS = 80.0;
```

즉, 버스 좌표에서 80m 안에 들어오는 최근 GPS 로그를 혼잡도 판단에 사용합니다.

처리 순서:

46. 버스를 DB 에서 찾습니다.
47. 현재 시각 기준 최근 15 분 GPS 로그를 가져옵니다.
48. 각 GPS 로그와 버스 사이 거리를 계산합니다.
49. 80m 안에 있는 로그만 남깁니다.
50. 최근 로그일수록 더 큰 가중치를 줍니다.
51. 가중치 합계를 반올림해 인원 수처럼 사용합니다.
52. 인원 수를 "여유", "보통", "혼잡", "매우혼잡"으로 바꿉니다.

단계 변환은 다음입니다.

```
private String convertLevel(int count) {
    if (count < 3) {
        return "여유";
    }
    if (count < 7) {
        return "보통";
    }
    if (count < 12) {
        return "혼잡";
    }
    return "매우혼잡";
}
```

24. SSE 실시간 구조

프론트는 api.js 에서 SSE 연결을 만듭니다.

```
export function createBoothStream() {
    return new EventSource(`${API_BASE}/stream/booths`);
}
```

백엔드는 StreamController 에서 구독 요청을 받습니다.

```
@GetMapping(value = "/booths", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public SseEmitter booths() {
    return streamService.subscribeBooths();
}
```

StreamService 는 연결 목록을 저장합니다.

```
private final List<SseEmitter> boothEmitters = new CopyOnWriteArrayList<>();
```

이후 어떤 서비스가 부스 변경을 발행하면:

```
public void publishBooths(Object payload) {
    send(boothEmitters, "booths", payload);
}
```

브라우저는 "booths" 이벤트를 받습니다.

SSE 채널은 다음과 같습니다.

경로	이벤트 이름	사용 예
`/api/stream/congestion`	`congestion`	혼잡도 갱신
`/api/stream/events`	`events`	공연 상태 갱신
`/api/stream/notices`	`notices`	공지 갱신
`/api/stream/booths`	`booths`	부스 상태/순서/이미지 갱신
`/api/stream/staff`	`staff`	스태프 상태/위치 갱신
`/api/stream/lost-items`	`lost-items`	분실물 갱신
`/api/stream/reservations`	`reservations`	예약 상태 갱신

25. 다른 주요 페이지 설명

25.1 `StageMapPage.jsx`

지도/부스 탐색 화면입니다.

주요 API:

- `fetchBooths()`
- `fetchAiVisitorGuide("stage-map")`

- createBoothStream()
- sendGps(latitude, longitude)

특징:

- 부스를 지도 마커로 표시합니다.
- 카테고리 필터와 검색을 제공합니다.
- 현재 위치 전송으로 GPS 로그를 백엔드에 저장합니다.
- 3 초마다 부스 목록을 새로고침합니다.
- AI 주변 부스 추천을 12 초마다 갱신합니다.

핵심 상수:

```
const BOOTH_REFRESH_INTERVAL_MS = 3000;
const AI_GUIDE_REFRESH_INTERVAL_MS = 12000;
```

25.2 `EventPage.jsx`

공연 일정 화면입니다.

주요 API:

- fetchEvents()
- fetchAiVisitorGuide("events")
- createEventStream()

특징:

- 날짜별 공연 탭을 만듭니다.
- 공연 상태를 표시합니다.
- 알림 저장은 localStorage 를 사용합니다.
- 공연 실시간 스트림으로 변경 사항을 반영합니다.

25.3 `AnalyticsPage.jsx`

혼잡도 분석 화면입니다.

주요 API:

- fetchAnalyticsDashboard(15)
- fetchAiCongestionPredictions()

- `fetchAiDecisionLogs()`
- `fetchAiVisitorGuide("analytics")`
- `createCongestionStream()`

특징:

- 전체 혼잡도 퍼센트
- 구역별 혼잡도
- 추세 차트
- AI 추천 행동
- 혼잡 예측 후보
- AI 판단 로그

25.4 `BoothDetailPage.jsx`

부스 상세/예약 화면입니다.

주요 API:

- `fetchBoothById(id)`
- `fetchCongestion(id)`
- `fetchBoothReservations(id, token)`
- `createBoothReservation(...)`
- `createBoothReservationCheckInToken(...)`
- `sendReservationAuthCode(phoneNumber)`
- `verifyReservationAuthCode(phoneNumber, code)`
- `createBoothStream()`
- `createReservationStream()`

특징:

- 부스 이미지, 소개, 메뉴, 운영 상태 표시
- 혼잡도 새로그침
- 전화번호 인증 후 예약
- 테이블 선택
- 체크인 QR 생성
- 부스 운영자 로그인 연결

25.5 `ChatPage.jsx`

AI 챗봇 화면입니다.

주요 API:

- askChat(question)

특징:

- 사용자 메시지와 AI 메시지를 대화 형태로 저장합니다.
- AI 근거를 표시합니다.
- 근거 타입에 따라 부스/공연/분실물 화면으로 이동할 수 있습니다.
- 경고 메시지가 있으면 요약해 보여줍니다.

25.6 `LostFoundPage.jsx`

분실물 화면입니다.

주요 API:

- fetchLostItems()
- createLostItem(form, file)
- createLostItemStream()

특징:

- 분실물 목록 조회
- 검색/상태 필터
- 분실물 등록
- 실시간 분실물 갱신

25.7 `MorePage.jsx`

더보기 메뉴 화면입니다.

특징:

- AI Match, 분실물, 챗봇, 스태프, 관리자 등 보조 기능 진입점입니다.
- 언어 설정 같은 부가 기능도 이곳에서 연결됩니다.

25.8 `AdminPage.jsx`

관리자 화면입니다.

주요 API:

- loginAdmin
- fetchBooths, createBooth, updateBooth, deleteBooth
- fetchEvents, createEvent, updateEvent, deleteEvent
- fetchAdminNotices, createNotice, updateNotice, deleteNotice
- fetchAdminDashboardKpis
- fetchAuditLogs
- fetchAdminStaff, updateAdminStaff
- CSV import/export

특징:

- JWT 로그인
- 부스/공연/공지 CRUD
- 부스 이미지 업로드
- 부스 순서 드래그 정렬
- 관리자 KPI
- 감사 로그
- 스태프 정보 관리

25.9 `OpsMasterPage.jsx`

통합 운영 콘솔입니다.

주요 API:

- fetchOpsMasterBootstrap(key)
- createOpsMasterNotice, updateOpsMasterNotice, deleteOpsMasterNotice
- createOpsMasterEvent, updateOpsMasterEvent, deleteOpsMasterEvent
- createOpsMasterBooth, updateOpsMasterBooth, deleteOpsMasterBooth
- fetchOpsMasterAiBriefing
- createOpsMasterAiNoticeDraft

특징:

- JWT 가 아니라 X-OPS-KEY 운영 키를 씁니다.

- 현장 운영자가 빠르게 공지/부스/공연을 바꿀 수 있게 만든 화면입니다.

25.10 `OpsBoothPage.jsx`

부스 운영자 전용 화면입니다.

주요 API:

- fetchOpsBoothBootstrap(boothId, key)
- updateOpsBoothLiveStatus
- uploadOpsBoothMenuImage
- fetchOpsBoothReservations
- updateOpsBoothReservationConfig
- checkInOpsBoothReservation
- completeOpsBoothReservation
- releaseOpsBoothReservationTable
- checkInOpsBoothReservationByToken

특징:

- 부스 하나의 운영 상태 관리
- 메뉴판/재고/대기 시간 갱신
- 예약 테이블 설정
- 예약 체크인/완료
- QR 체크인 처리

25.11 `StaffPage.jsx`

스태프 콘솔입니다.

주요 API:

- loginStaff
- fetchStaffBootstrap
- updateMyStaffStatus
- createStaffStream
- fetchLostItems
- createLostItem
- updateLostItem
- updateLostItemStatus
- deleteLostItem
- fetchStaffAiZoneSummary

- createStaffAiLostItemAssist
- fetchStaffAiFieldChecklist
- translateText

특징:

- 스태프 번호 + PIN 로그인
- 상태와 위치 갱신
- 분실물 등록/수정/반환 상태 관리
- AI 현장 체크리스트
- AI 분실물 응대
- 실시간 번역

26. 브라우저 저장소 구조

프론트엔드는 일부 값을 브라우저 localStorage 에 저장합니다. 서버 DB 와 헛갈리면 안 됩니다.

26.1 `utils/storage.js`

key	의미
`festflow_favorites`	즐거찾기 부스 ID 목록
`festflow_recents`	최근 본 부스 ID 목록
`festflow_memos`	부스별 개인 메모

예를 들어 즐거찾기 토글은 다음 방식입니다.

```
export function toggleFavorite(id) {
  const favorites = new Set(getFavoriteIds());
  if (favorites.has(id)) {
    favorites.delete(id);
  } else {
    favorites.add(id);
  }
  localStorage.setItem(FAVORITES_KEY, JSON.stringify([...favorites]));
  return [...favorites];
}
```

26.2 `utils/reservationAuth.js`

key	의미
`festflow_reservation_auth_token`	전화번호 인증 후 받은 예약 토큰
`festflow_reservation_phone`	인증된 전화번호

예약 기능은 이 토큰을 백엔드에 보내서 사용자를 식별합니다.

26.3 `utils/auth.js`

key	의미
`festflow_access_token`	관리자 JWT
`festflow_admin_name`	관리자 이름

관리자 API 는 이 JWT 를 Authorization 헤더에 붙여 호출합니다.

27. 스타일 구조: `index.css`

파일: frontend/src/index.css

이 프로젝트는 Tailwind 도 쓰지만, 실제 화면 스타일 상당 부분은 index.css 에 직접 정의되어 있습니다.

27.1 앱 전체 껍데기

```
.app-shell {
  position: relative !important;
  width: 100% !important;
  max-width: 390px !important;
  min-height: 100dvh !important;
  margin: 0 auto !important;
  overflow: hidden !important;
  background: ...;
}
```

이 설정 때문에 앱은 모바일 앱처럼 최대 너비 390px 중심 화면으로 보입니다. 데스크톱에서 열어도 가운데에 모바일 기기 화면처럼 표시됩니다.

27.2 홈 히어로 카드

```
.home-hero-card {
  min-height: 21.2rem;
  display: flex;
  flex-direction: column;
  justify-content: space-between;
  padding: 1.05rem;
  border-radius: 1.25rem;
  background-position: center;
  background-size: cover;
  color: #ffffff;
}
```

이 스타일은 홈 상단의 큰 카드입니다.

- `min-height`: 카드 높이 확보
- `display: flex`: 내부 요소 배치
- `justify-content: space-between`: 위/아래 요소 간격 분산
- `border-radius`: 둥근 모서리
- `background-size: cover`: 배경 이미지/그라디언트가 카드 전체를 덮게 함

27.3 추천 카드

```
.recommend-strip {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 0.55rem;
}
```

추천 카드 3 개를 같은 너비로 배치합니다.

28. i18n 구조: `i18n.js`

파일: `frontend/src/i18n.js`

이 파일은 한글 화면을 영어로 바꾸기 위한 간단한 번역 시스템입니다.

핵심 개념:

- 53. KO_TO_EN 객체에 한글과 영어 매핑을 둡니다.
- 54. LanguageProvider 가 현재 언어를 state 로 관리합니다.
- 55. MutationObserver 가 DOM 변경을 감지합니다.
- 56. 텍스트 노드와 일부 속성(placeholder, aria-label, title, alt)을 번역합니다.

주의할 점:

- 이 방식은 React 렌더 후 DOM 텍스트를 바꾸는 방식입니다.
- 일반적인 i18n 라이브러리처럼 컴포넌트마다 t("...")를 강제하지 않습니다.
- 동적으로 생기는 텍스트도 MutationObserver 가 다시 번역하려고 시도합니다.

29. 백엔드 설정 파일

대표 설정은 backend/src/main/resources/application.properties 입니다.

중요 설정:

설정	의미
`spring.datasource.url`	DB 주소
`spring.datasource.username`	DB 사용자
`spring.datasource.password`	DB 비밀번호
`spring.jpa.hibernate.ddl-auto`	JPA schema 처리 방식
`server.port`	백엔드 포트
`app.jwt.secret`	JWT secret
`app.upload.dir`	로컬 업로드 폴더
`app.storage.type`	`local` 또는 `s3`
`app.cors.allowed-origins`	허용할 프론트 주소
`app.ops.master-key`	통합 운영 키
`app.ops.booth-keys`	부스 운영 키
`app.sms.provider`	SMS provider

`app.openai.api-key`	OpenAI API 키
`app.openai.model`	AI 텍스트 모델
`app.openai.image-model`	AI 이미지 모델

로컬 기본 API base 는 프론트에서 `http://localhost:8080/api` 입니다.

30. API 전체 요약

30.1 방문자 API

메서드	경로	설명
GET	`/api/booths`	부스 목록
GET	`/api/booths/{id}`	부스 상세
GET	`/api/booths/{id}/congestion`	특정 부스 혼잡도
GET	`/api/booths/{id}/reservations`	예약 상태
POST	`/api/booths/{id}/reservations`	예약 생성
POST	`/api/booths/{id}/reservations/{reservationId}/check-in-token`	QR 체크인 토큰 생성
GET	`/api/events`	공연 목록
POST	`/api/gps`	GPS 위치 로그 저장
POST	`/api/chat`	AI 챗봇 질문
GET	`/api/notices/active`	활성 공지
GET	`/api/lost-items`	분실물 목록
POST	`/api/lost-items`	분실물 등록
PUT	`/api/lost-items/{id}/claim`	내 물건 표시
POST	`/api/reservations/auth/send-code`	예약 전화번호 인증번호 발송
POST	`/api/reservations/auth/verify-code`	예약 인증번호 확인

30.2 분석/AI API

메서드	경로	설명
GET	`/api/analytics/traffic-hourly`	최근 24 시간 시간대별 방문량
GET	`/api/analytics/popular-booths`	인기 부스
GET	`/api/analytics/congestion-heatmap`	혼잡 히트맵
GET	`/api/analytics/stage-crowd`	무대 혼잡
GET	`/api/analytics/dashboard`	혼잡 대시보드
GET	`/api/ai/guide`	AI 축제 가이드
GET	`/api/ai/congestion/predictions`	AI 혼잡 예측
GET	`/api/ai/decisions`	AI 판단 로그
GET	`/api/ai/visitor-guide/{scope}`	페이지별 방문자 AI 가이드

30.3 관리자 API

메서드	경로	설명
POST	`/api/auth/login`	관리자 로그인
GET	`/api/admin/dashboard/kpis`	관리자 KPI
GET	`/api/admin/audit-logs`	감사 로그
POST/PUT/DELETE	`/api/admin/booths/**`	부스 관리
POST/PUT/DELETE	`/api/admin/events/**`	공연 관리
GET/POST/PUT/DELETE	`/api/admin/notices/**`	공지 관리
POST	`/api/admin/import/booths`	부스 CSV import
POST	`/api/admin/import/events`	공연 CSV import
GET/PUT	`/api/admin/staff/**`	스태프 관리

30.4 운영 API

운영 API 는 X-OPS-KEY 헤더를 사용합니다.

메서드	경로	설명
GET	`/api/ops/master/bootstrap`	통합 운영 초기 데이터
POST/PUT/DELETE	`/api/ops/master/notices/**`	운영 공지 관리
POST/PUT/DELETE	`/api/ops/master/events/**`	운영 공연 관리
POST/PUT/DELETE	`/api/ops/master/booths/**`	운영 부스 관리
POST	`/api/ops/master/ai/briefing`	AI 운영 브리핑
POST	`/api/ops/master/ai/notice-draft`	AI 공지 초안
GET	`/api/ops/booth/{id}/bootstrap`	부스 운영 초기 데이터
PUT	`/api/ops/booth/{id}/live-status`	부스 운영 상태 저장
POST	`/api/ops/booth/{id}/menu-image`	메뉴 이미지 업로드
GET	`/api/ops/booth/{id}/reservations`	예약 대시보드
PUT	`/api/ops/booth/{id}/reservations/config`	예약 설정 저장
POST	`/api/ops/booth/{id}/reservations/{reservationId}/check-in`	예약 체크인
POST	`/api/ops/booth/{id}/reservations/{reservationId}/complete`	이용 완료

31. DB Entity 큰 그림

Entity	의미
`Booth`	부스 기본 정보, 좌표, 운영 상태, 이미지, 예약 가능 여부
`FestivalEvent`	공연 일정과 상태
`GpsLog`	혼잡도 계산용 위치 로그
`Notice`	공지
`LostItem`	분실물

`AdminUser`	관리자 계정
`AuditLog`	관리자 작업 로그
`StaffMember`	스태프 계정과 현장 상태
`StaffSession`	스태프 로그인 세션
`BoothReservationTable`	부스별 테이블
`BoothReservation`	예약
`ReservationCheckInToken`	예약 체크인 QR 토큰
`ReservationUserAccount`	예약 전화번호 사용자
`ReservationAuthSession`	예약 인증 세션
`ReservationVerificationCode`	인증번호
`ReservationUserState`	노쇼/차단 상태
`AiMatchProfile`	AI Match 프로필
`AiMatchRequest`	AI Match 신청
`AiMatchFavorite`	AI Match 좋아요
`AiMatchPhoneUsage`	AI Match 전화번호 사용량

32. 인증과 권한

FestFlow 에는 여러 종류의 인증이 있습니다.

영역	인증 방식	사용 위치
일반 방문자	대부분 인증 없음	홈, 지도, 공연, 챗봇
예약 사용자	전화번호 인증 토큰	부스 예약
관리자	JWT Bearer 토큰	`/api/admin/**`
통합 운영자	`X-OPS-KEY`	`/api/ops/master/**`
부스 운영자	`X-OPS-KEY`	`/api/ops/booth/{id}/**`
스태프	`X-Staff-Token`	`/api/staff/**`

AI Match 사용자	닉네임 + PIN	프로필 수정/삭제, 신청 처리
--------------	-----------	------------------

관리자 토큰은 프론트의 `utils/auth.js` 를 통해 `localStorage` 에 저장됩니다.

운영 키는 JWT 와 다르게 단순 헤더 기반입니다. 그래서 운영 키는 외부에 노출되면 안 됩니다.

33. PWA 구조

PWA 관련 파일은 `frontend/public` 에 있습니다.

파일	역할
<code>`manifest.json`</code>	앱 이름, 아이콘, 설치 설정
<code>`service-worker.js`</code>	캐싱과 오프라인 처리
<code>`offline.html`</code>	오프라인 상태 페이지
<code>`pwa-192.png`</code> , <code>`pwa-512.png`</code>	PWA 아이콘
<code>`apple-touch-icon.png`</code>	iOS 홈 화면 아이콘

`main.jsx`에서는 `production` 환경일 때 `service worker` 를 등록합니다.

```
if (import.meta.env.PROD && "serviceWorker" in navigator) {
  window.addEventListener("load", () => {
    navigator.serviceWorker.register("/service-worker.js").catch(() => {
      // Service worker registration failure should not block the app.
    });
  });
}
```

즉, 개발 서버에서는 보통 `service worker` 가 등록되지 않고, 배포 빌드에서만 등록됩니다.

34. 배포 구조

README 기준 배포 방향은 다음과 같습니다.

영역	플랫폼	설정
----	-----	----

백엔드	Railway	`backend` 폴더를 서비스 루트로 설정
프론트엔드	Vercel	`frontend` 폴더를 Root Directory 로 설정
DB	MySQL	`SPRING_DATASOURCE_*` 환경변수 연결
API 연결	Vercel env	`VITE_API_BASE_URL=https://백엔드도메인/api`

프론트 배포 설정:

- Build Command: `npm run build`
- Output Directory: `dist`
- React Router fallback: `frontend/vercel.json`

백엔드 운영 필수 환경변수:

- `SPRING_DATASOURCE_URL`
- `SPRING_DATASOURCE_USERNAME`
- `SPRING_DATASOURCE_PASSWORD`
- `SPRING_PROFILES_ACTIVE=prod`
- `APP_JWT_SECRET`
- `APP_CORS_ALLOWED_ORIGINS`
- `APP_INIT_ADMIN_USERNAME`
- `APP_INIT_ADMIN_PASSWORD`

AI 기능을 쓰려면:

- `OPENAI_API_KEY`
- `OPENAI_MODEL`
- `OPENAI_IMAGE_MODEL`

SMS 인증을 쓰려면 provider 별 키가 필요합니다.

35. 자주 만나는 오류

35.1 화면에 "서버에 연결할 수 없습니다"가 보임

가능한 원인:

- 백엔드 서버가 꺼져 있음
- `VITE_API_BASE_URL` 이 잘못됨

- CORS 설정이 맞지 않음
- 네트워크가 막힘

확인할 것:

```
백엔드: http://localhost:8080
프론트 API base: frontend/src/api.js 또는 VITE_API_BASE_URL
브라우저 개발자도구 Network 탭
```

35.2 홈 화면이 fallback 데이터만 보여줌

가능한 원인:

- /api/booths 실패
- /api/events 실패
- DB 에 데이터 없음
- 백엔드 초기 데이터 삽입 실패

홈 화면은 실패해도 fallbackBooths, fallbackEvents 를 사용하므로 완전히 빈 화면이 되지는 않습니다.

35.3 AI 답변이 기본 안내로만 나옴

가능한 원인:

- OPENAI_API_KEY 가 설정되지 않음
- OpenAI API 호출 실패
- 질문과 연결되는 축제 근거 데이터를 찾지 못함

ChatService 는 API 키가 없으면 fallback 답변을 만듭니다.

35.4 관리자 API 가 401 또는 403

가능한 원인:

- 로그인하지 않음
- JWT 만료

- APP_JWT_SECRET 설정 문제
- Authorization 헤더 누락

프론트에서는 `withAuth()`가 토큰을 붙입니다. `localStorage` 에 토큰이 있는지 확인해야 합니다.

35.5 예약이 안 됨

가능한 원인:

- 전화번호 인증 토큰 없음
- 이미 활성 예약이 있음
- 좌석이 부족함
- 노쇼 패널티로 차단됨
- 예약 가능한 테이블이 없음

관련 파일:

- `frontend/src/pages/BoothDetailPage.jsx`
- `frontend/src/utis/reservationAuth.js`
- `backend/src/main/java/com/festflow/backend/service/ReservationService.java`
- `backend/src/main/java/com/festflow/backend/service/ReservationAuthService.java`

35.6 실시간 갱신이 안 됨

확인할 것:

- 브라우저 Network 탭에서 `/api/stream/...` 연결이 유지되는지
- 백엔드 `StreamService.publish...`가 호출되는지
- 프론트가 올바른 이벤트 이름을 듣고 있는지

예를 들어 부스 스트림은 이벤트 이름이 "booths"입니다. 프론트가 다른 이름을 들으면 갱신되지 않습니다.

36. 개발자가 수정할 때 체크리스트

36.1 새 페이지 추가

- 57. frontend/src/pages/NewPage.jsx 생성
- 58. frontend/src/main.jsx 에서 lazy import 추가
- 59. Routes 에 `<Route path="..." element={lazyElement(NewPage)} />` 추가
- 60. 필요하다면 App.jsx 의 하단 네비게이션 match 수정
- 61. 스타일이 필요하다면 index.css 추가

36.2 새 API 추가

- 62. 백엔드 DTO 생성
- 63. Controller 에 API 경로 추가
- 64. Service 에 실제 로직 추가
- 65. Repository 조회가 필요하다면 메서드 추가
- 66. 프론트 api.js 에 호출 함수 추가
- 67. 페이지 컴포넌트에서 호출
- 68. 권한이 필요한 경우 Security 설정 확인
- 69. 상태 변경이 실시간으로 보여야 하면 StreamService 발행 추가

36.3 새 DB 필드 추가

- 70. Entity 에 필드 추가
- 71. DTO 에 필드 추가
- 72. Service 의 toDto 변환 수정
- 73. 생성/수정 요청 DTO 수정
- 74. 프론트 api.js payload 확인
- 75. 화면 입력/표시 추가
- 76. 운영 배포에서는 schema migration 또는 DB 컬럼 추가 필요

36.4 실시간 값 추가

- 77. 백엔드 상태 변경 Service 에서 DB 저장
- 78. `StreamService.publish...` 호출
- 79. `StreamController` 에 구독 경로가 없으면 추가
- 80. `api.js` 에 `create...Stream` 추가
- 81. 프론트 페이지에서 `EventSource` 구독
- 82. `cleanup` 에서 `stream.close()` 호출

37. 코드 읽기 연습: 홈 화면 데이터 흐름을 직접 추적하기

초보자는 다음 순서대로 실제 파일을 열어보면 됩니다.

37.1 `/`가 왜 홈 화면인가?

- 83. `frontend/src/main.jsx` 열기
- 84. 아래 코드 찾기

```
<Route index element={lazyElement(HomePage)} />
```

- 85. 이것이 / 경로가 `HomePage` 라는 뜻입니다.

37.2 홈 화면이 부스를 어디서 가져오나?

- 86. `frontend/src/pages/HomePage.jsx` 에서 `fetchBooths()` 찾기
- 87. `frontend/src/api.js` 에서 `fetchBooths` 함수 찾기
- 88. `backend/src/main/java/com/festflow/backend/controller/BoothController.java` 에서 `@GetMapping` 찾기
- 89. `backend/src/main/java/com/festflow/backend/service/BoothService.java` 에서 `getAllBooths` 찾기

37.3 홈 화면이 실시간으로 바뀌는 이유는?

- 90.HomePage.jsx 에서 createBoothStream() 찾기
- 91.api.js 에서 createBoothStream 찾기
- 92.StreamController.java 에서 /booths 찾기
- 93.StreamService.java 에서 publishBooths 찾기
- 94.부스 상태 변경 서비스에서 publishBooths 호출 위치 찾기

38. 핵심 요약

FestFlow 홈페이지를 마스터하려면 다음 다섯 가지를 이해하면 됩니다.

- 95.main.jsx 가 URL 과 페이지를 연결합니다.
- 96.App.jsx 가 공통 레이아웃과 하단 메뉴를 담당합니다.
- 97.HomePage.jsx 가 첫 화면의 데이터 로딩, AI 질문, 추천 카드, 실시간 스트림을 담당합니다.
- 98.api.js 가 프론트와 백엔드 사이의 모든 HTTP/SSE 연결을 모아둡니다.
- 99.백엔드는 Controller 가 요청을 받고 Service 가 실제 규칙을 처리하며 Repository/Entity 가 DB 와 연결됩니다.

가장 중요한 실제 흐름은 이 한 줄입니다.

React 화면의 사용자 행동 -> api.js 함수 -> Spring Controller -> Service 로직 -> DB/SSE/AI/SMS -> React state 갱신 -> 화면 변경

이 구조를 잡고 나면, 홈 화면뿐 아니라 지도, 공연, 혼잡도, 예약, 관리자, 운영, 스태프, AI Match 까지 같은 방식으로 읽을 수 있습니다.

부록 A. 핵심 코드 상세 해부

이 부록의 목적

앞 장들이 전체 구조를 설명했다면, 이 부록은 실제 코드 몇 줄이 무슨 의미인지 초보자 눈높이로 다시 풀어 설명합니다. 코드를 직접 열어놓고 같이 보면 가장 효과적입니다.

A.1 HomePage 가 처음 렌더링될 때

```
useEffect(() => {
  let mounted = true;

  Promise.allSettled([
    fetchBooths(),
    fetchEvents(),
    fetchTrafficHourly(),
  ]).then(([boothResult, eventResult, trafficResult]) => {
    if (!mounted) return;
    ...
  });

  return () => {
    mounted = false;
    streams.forEach((stream) => stream.close());
  };
}, []);
```

이 코드는 홈 화면 생명주기의 핵심입니다. 빈 의존성 배열 [] 때문에 화면이 처음 나타날 때 한 번만 실행됩니다. `Promise.allSettled` 는 부스, 공연, 방문량 요청을 동시에 보내고, 실패한 요청이 있어도 성공한 요청 결과를 사용할 수 있게 합니다. `return` 안의 함수는 화면이 사라질 때 실행되는 정리 함수이며, SSE 연결을 닫아 메모리 누수와 중복 이벤트 수신을 막습니다.

A.2 api.js 에서 URL 을 안정적으로 만드는 방식

```
const API_BASE = (
  import.meta.env.VITE_API_BASE_URL || "http://localhost:8080/api"
)
  .trim()
  .replace(/\/+$/, "");
```

`import.meta.env.VITE_API_BASE_URL` 은 Vite 가 제공하는 환경변수 접근 방식입니다. 운영 배포에서는 Railway 백엔드 주소를 여기에 넣고, 로컬에서는 값이 없으므로 `http://localhost:8080/api` 가 사용됩니다. 마지막의 `replace(/\/+$/, "")` 는 주소 끝의 `/` 를 제거해 `/api//booths` 같은 잘못된 URL 을 방지합니다.

A.3 백엔드 Controller 가 얇은 이유

```
@GetMapping
public List<BoothResponseDto> getBooths() {
    return boothService.getAllBooths();
}
```

Controller 는 HTTP 요청을 받는 입구입니다. 여기서 DB 조회, 정렬, 혼잡도 계산을 직접 하지 않고 `BoothService` 로 넘깁니다. 이렇게 하면 API 주소를 담당하는 코드와 실제 도메인 규칙을 담당하는 코드가 분리되어 유지보수가 쉬워집니다.

A.4 Service 에서 DTO 로 바꾸는 이유

Entity 는 DB 테이블과 가깝고, DTO 는 API 응답과 가깝습니다. Entity 를 그대로 프론트에 보내면 내부 필드나 민감한 정보가 노출될 수 있고, 화면에 필요한 계산값을 붙이기 어렵습니다. 그래서 `BoothService.toDto` 처럼 서버 내부 모델을 화면용 응답 모델로 변환합니다.

A.5 SSE 이벤트 이름이 중요한 이유

```
emitter.send(SseEmitter.event().name(eventName).data(payload));

boothStream.addListener("booths", (event) => {
    const next = JSON.parse(event.data);
    setBooths(next);
});
```

백엔드가 `.name("booths")` 로 이벤트를 보내면 프론트도 정확히 `addListener("booths", ...)` 로 들어야 합니다. 경로가 맞아도 이벤트 이름이 다르면 콜백이 실행되지 않습니다. 실시간 갱신 문제를 디버깅할 때는 경로와 이벤트 이름을 모두 확인해야 합니다.